Stage 3: Timestamp Alignment

Theory notes and interview questions
Project: Doctor–Patient Conversation Audio -> SOAP Notes

What this stage does: combines Whisper's words (what + when) with pyannote's speaker intervals (who + when) to produce a speaker-labelled transcript. No model is run - it is pure interval arithmetic on two JSON files. Speakers stay anonymous (SPEAKER_00 / SPEAKER_01).

Contents

1. The problem: two independent timelines
2. Are Whisper's word timestamps reliable enough?
3. Forced alignment (wav2vec2 / WhisperX) - the upgrade path
4. Interval overlap
5. The assignment rules
6. Boundary cases with worked examples
7. Why exclusive_speakers
8. Grouping words into utterances
9. Our code (align.py)
10. Reading our output
11. Complexity
12. Evaluation: WDER and cpWER
13. Limitations
14. Interview questions and answers

1. The problem: two independent timelines

```
Audio
■■■ Whisper (Stage 1)   -> words + timestamps        'what was said, when'
■■■ pyannote (Stage 2)  -> speaker intervals          'who spoke, when'
                  \           /
            Timestamp alignment (Stage 3)
                      |
            Speaker-labelled transcript              'who said what'
```

- The two models never talk to each other. The **only** thing they share is the audio clock, so time is the key used to join them - like a database join on time ranges.
- Their boundaries never match exactly: e.g. Whisper says 'today?' ends at 2.08 s, pyannote says SPEAKER_00 talks until 2.46 s. Different frame rates (20 ms vs ~17 ms), different training data and different ideas of where a sound fades out.
- So we cannot ask 'is this word exactly inside a turn?' - we need a tolerant rule. That rule is **maximum temporal overlap**.

2. Are Whisper's word timestamps reliable enough?

faster-whisper estimates word times from cross-attention + Dynamic Time Warping (see Stage 1 notes). We measured them against ground truth: in our synthetic clip the silences between turns are digital silence, so the true start and end of each turn can be read from the signal energy.

Turn	True sound	Whisper words	pyannote	Whisper error
1	0.00 – 2.39	0.00 – 2.08	0.03 – 2.46	start 0.00, end -0.31
2	2.91 – 6.49	2.84 – 6.42	2.88 – 6.54	-0.07 / -0.07
3	7.09 – 9.24	7.00 – 9.16	7.07 – 9.35	-0.09 / -0.08
4	9.76 – 13.69	9.70 – 13.60	9.70 – 13.78	-0.06 / -0.09

- Typical error under 0.1 s, slight early bias; worst case 0.31 s on a drawn-out question intonation.
- 33/33 words overlap a pyannote turn. Errors are much smaller than a typical word (0.2-0.5 s), so maximum overlap still picks the right speaker.
- **Decision:** keep Whisper's timestamps; no new model. The engineering principle: measure first, add complexity only when the data shows a need.

3. Forced alignment (wav2vec2 / WhisperX) - the upgrade path

- **Forced alignment** = given audio AND its known transcript, find exactly when each word/phoneme occurs. Easier than recognition because the words are already known.
- WhisperX uses a **wav2vec2** model fine-tuned with **CTC** (Connectionist Temporal Classification): it outputs, every ~20 ms, a probability for each character/phoneme plus a 'blank'.
- A **Viterbi** search finds the most likely path through those probabilities that spells the known transcript in order; the frames where each word's characters are emitted give its start/end (about 20-50 ms precision).
- It works **alongside** Whisper: Whisper decides the words, wav2vec2 refines only their timing. Output format stays {word, start, end}, so align.py would not change.

- When to add it: if real recordings show words misassigned at fast speaker changes (short 'yes', 'mm-hm').

4. Interval overlap

```
overlap = max(0, min(word_end, speaker_end) - max(word_start, speaker_start))

word      :        [1.20 ======== 1.80]
SPEAKER_00: [0.03 ========= 1.50]
SPEAKER_01:                 [1.50 ============ 4.00]
  overlap with 00 = min(1.80,1.50) - max(1.20,0.03) = 1.50 - 1.20 = 0.30
  overlap with 01 = min(1.80,4.00) - max(1.20,1.50) = 1.80 - 1.50 = 0.30   -> tie
```

- min of the ends = where the shared part stops; max of the starts = where it begins.
- If they don't touch, that difference is negative, so max(0, ...) clamps it to 0.
- If a speaker has several intervals touching the same word, their overlaps are **summed per speaker**.

5. The assignment rules (in order)

#	Situation	Rule	Label in JSON
1	Word overlaps one or more speakers	Speaker with the largest total overlap	overlap
2a	Tie for largest overlap	Speaker whose interval contains the word midpoint	tie-midpoint
2b	Tie, midpoint decides nothing	Speaker of the previous word (people rarely switch mid-word)	tie-previous
2c	Still undecided	Tied speaker whose interval starts first	tie-first
3	No overlap (word in a gap)	Nearest interval if within 0.5 s (--max-gap)	nearest
4	Nothing within 0.5 s	UNKNOWN - reported, never guessed	none

Intervals are treated as **half-open [start, end)**: a moment exactly on a boundary belongs to the speaker who starts there. That makes the midpoint rule deterministic - a midpoint can never be in two intervals.

6. Boundary cases with worked examples (tested)

Word (s)	Speaker intervals	Result	Why
Hello 0.10-0.40	00: 0.03-1.50	SPEAKER_00 (overlap)	only overlap, 0.30 s
cough 1.30-1.90	00: 0.03-1.50, 01: 1.50-4.00	SPEAKER_01 (overlap)	0.40 s vs 0.20 s
fever 1.20-1.80	00: 0.03-1.50, 01: 1.50-4.00	SPEAKER_01 (tie-midpoint)	0.30 = 0.30; midpoint 1.50 is the start of 01's [1.50, 4.00)

Word (s)	Speaker intervals	Result	Why
um 4.10-4.30	01 ends 4.00, next turn 6.00	SPEAKER_01 (nearest)	in a gap, 0.10 s from 01
noise 4.80-5.20	01 ends 4.00, next turn 6.00	UNKNOWN (none)	0.80 s from nearest turn > 0.5 s

7. Why exclusive_speakers

- Regular diarization allows two speakers at once. Whisper, however, outputs **one** stream of words. During overlap a word would have two plausible owners and the transcript would duplicate or split oddly.
- Exclusive diarization keeps only the most likely speaker at each moment, so the timeline is a clean partition - exactly what a single transcript needs.
- Trade-off: in real overlaps the quieter speaker's words (if Whisper caught them) are attributed to the dominant speaker. Regular diarization is kept in the JSON for later analysis.

8. Grouping words into utterances

- Walk through words in time order; while the speaker stays the same, append the word to the current utterance; when it changes, start a new one.
- Utterance start = first word's start, end = last word's end (word times, not diarization times - they describe the actual spoken text).
- Whisper already attaches punctuation to words ('today?'), so joining with spaces restores the sentence.
- Note: utterances follow **speakers**, not Whisper segments. A Whisper segment containing a speaker change is split; two segments by the same speaker in a row are merged.

9. Our code (align.py)

Function	Purpose
overlap(a_start, a_end, b_start, b_end)	The overlap formula.
distance(word, interval)	Gap between a word and an interval (0 if overlapping) - used by rule 3.
assign_speaker(word, intervals, previous, max_gap)	Applies rules 1-4; returns (speaker, method, overlap).
align_words(words, intervals, max_gap)	Runs assign_speaker on every word, tracking the previous speaker.
group_utterances(aligned_words)	Merges consecutive same-speaker words.
main()	Loads outputs/<name>.json and <name>_diarization.json, prints, saves <name>_aligned.json with a summary of methods used.

No model is loaded and nothing is installed: transcribe.py and diarize.py are unchanged and their JSON files are the inputs. Each word in the output carries **assignment** and **overlap** so every decision is auditable - important in a medical setting.

10. Reading our output

```
[00:00.00 → 00:02.08]  SPEAKER_00: Good morning, what brings you in today?
[00:02.84 → 00:06.42]  SPEAKER_01: I have been experiencing fever for three days, and a dry cough.
[00:07.00 → 00:09.16]  SPEAKER_00: Have you taken any medication for it?
[00:09.70 → 00:13.60]  SPEAKER_01: Just paracetamol, 500 milligrams, twice a day.
Words: 33  |  by method: {'overlap': 33}
```

- All 33 words were decided by rule 1 (clear overlap); no ties, gaps or unknowns - expected for a clean clip with silence between turns.
- Every word went to the correct voice (Daniel = SPEAKER_00, Samantha = SPEAKER_01).
- Real recordings will exercise the fallback rules; the summary line tells you how often.

11. Complexity

Our implementation compares every word with every interval: **O(W x S)** for W words and S intervals. A 15-minute consultation has about 2,000 words and 300 turns = 600,000 cheap comparisons: instant. For hours of audio, since both lists are sorted by time, a **two-pointer sweep** gives O(W + S), or an interval tree gives O((W + S) log S).

12. Evaluation: WDER and cpWER

- **WDER** (Word Diarization Error Rate): fraction of correctly recognised words assigned to the wrong speaker. Directly measures this stage.
- **cpWER** (concatenated minimum-permutation WER): concatenate each speaker's words, find the best mapping of system to reference speakers, compute WER. Penalises both recognition and speaker errors.
- DER (Stage 2) measures time, not words; a pipeline can have good DER and still misattribute key short words.

13. Limitations

- **Overlapping speech**: Whisper usually transcribes only one speaker; the other's words are lost.
- **Short turns at speaker changes** ('yes', 'mm-hm') are the most likely to be misassigned because a 0.1 s timestamp error is large relative to the word.
- **Errors compound**: a diarization confusion or a hallucinated word passes straight into the transcript.
- **Fixed 0.5 s gap tolerance** is a heuristic; it should be tuned on real annotated data.
- Labels remain anonymous - deciding who is the doctor is Stage 4.

14. Interview questions and answers

Q1. What is timestamp alignment in a diarization pipeline?

Joining the ASR output (words with times) and the diarization output (speaker intervals) on the shared time axis, so each word gets a speaker label, producing a 'who said what' transcript.

Q2. Why not just label each Whisper segment with a speaker?

A segment can contain a speaker change (interruptions, quick answers). Word-level assignment lets a segment be split across speakers; segment-level assignment would mislabel part of it.

Q3. Write the overlap formula and explain it.

max(0, min(end1, end2) - max(start1, start2)). The shared part starts at the later start and ends at the earlier end; if that is negative the intervals don't touch, so clamp to 0.

Q4. Why maximum overlap rather than the word midpoint?

It uses the whole word duration, so it is more tolerant of small timestamp errors. Midpoint is a single point that can flip on a 0.1 s error; we use it only as a tie-breaker.

Q5. How do you break ties?

Midpoint containment with half-open intervals; then the previous word's speaker; then the earliest-starting tied interval. Deterministic, documented, and recorded per word in the JSON.

Q6. What do you do with words that fall in a gap between turns?

Assign the nearest interval if within 0.5 s and flag it 'nearest'; otherwise mark UNKNOWN and report it. Never silently guess - silent errors are worse than visible ones in clinical data.

Q7. Why use exclusive diarization for alignment?

Whisper produces one word stream; exclusive diarization gives a non-overlapping timeline, so each moment has a single owner. Trade-off: in true overlap the quieter speaker is not represented.

Q8. How did you decide whether Whisper's word timestamps were good enough?

Measured them against ground-truth turn boundaries from signal energy on a clip with digital silence between turns: errors under 0.1 s typically, 0.31 s worst case, and all words overlapped a turn. Good enough for max-overlap assignment, so no extra model.

Q9. What is forced alignment and when would you add it?

Locating known transcript words in audio using a CTC acoustic model (e.g. wav2vec2) and Viterbi decoding, with ~20-50 ms precision. Add it if real data shows misassignment of short words at speaker changes.

Q10. Does WhisperX replace Whisper?

No. It runs Whisper for transcription, then wav2vec2 for alignment, then optionally pyannote for diarization and a word-to-speaker assignment similar to ours.

Q11. What is CTC?

Connectionist Temporal Classification: a training loss/output scheme where the model emits a label or a 'blank' per frame, allowing alignment-free training; the frame-wise probabilities can later be used for alignment.

Q12. What is the time complexity of your alignment and how would you scale it?

O(W x S). Because both lists are time-sorted, a two-pointer sweep makes it O(W + S); an interval tree gives O((W+S) log S).

Q13. How would you evaluate this stage?

WDER on annotated consultations (fraction of words with wrong speaker), plus cpWER for end-to-end speaker-attributed accuracy. Also track how many words used fallback rules.

Q14. What happens with overlapping speech in your pipeline?

Whisper typically transcribes the dominant speaker; exclusive diarization assigns overlapping time to one speaker; the other speaker's words may be lost or merged. A known limitation.

Q15. Why store 'assignment' and 'overlap' for every word?

Auditability and debugging: you can see how each decision was made, filter low-confidence assignments, and measure how often fallbacks are used on real data.

Q16. Why are utterance times taken from words, not diarization?

They should describe the spoken text. Diarization intervals include trailing silence/fade (e.g. 2.46 vs 2.08 s) and may cover several words not in that utterance.

Q17. Could the speaker labels be mapped to doctor/patient here?

Not reliably from acoustics alone; labels are arbitrary per recording. Role identification is a separate stage using content cues (questions, medical terminology) or enrollment.